

Week 2 — Industry-Graded Minor Project

SpendDNA

your wallet's year-end story.

"Spotify Wrapped for your money."

INDUSTRY CONNECT

Why this project, and why nothing else.

GroupDNA decoded your social fingerprint. SpendDNA decodes your financial fingerprint. Together they form the Unlox DNA series - two LinkedIn-grade projects that nobody else in Indian coaching is building.

Every Indian student opens a UPI app at least 5 times a day. Every one of them has thought, at least once, "where did all my money go this month?" There has never been a more universally relatable problem to solve in Python - and there is currently no tool that gives the student a real answer. By the end of this week, you will have built that tool. You will analyse 6 months of synthetic Indian transaction data, extract vendors from messy descriptions, categorise spending across 12 categories, detect personality archetypes, flag anomalies, and produce a printed report that any 22-year-old fintech analyst at Cred or Slice would be proud to ship.

You are not building a project. You are building an audition for your first fintech job.

Difficulty: Industry-grade **Time:** 7 days

Concepts: Strings, Lists, Dicts, Sets, Functions, NumPy, Pandas, Datetime, EDA, Anomaly Detection **Output:** One .ipynb notebook, submitted as Colab % GitHub link.

1. The Vision - What You're Actually Building

Imagine this. You finish the project. You run your code on your own bank statement export. The output is a beautiful, formatted analytics report that tells you exactly where your money went over the last 6 months. What percentage of your spending was on Swiggy. Which restaurants you ordered from most. How much you spent on Zepto late at night. What your financial personality archetype is.

You screenshot it. You post it on LinkedIn with the caption: "I spent a weekend building a Python tool that analyzes 6 months of my own UPI transactions. Found out 31% of my spending went to Swiggy and Zomato - and 72% of those orders happened between 9 PM and 1 AM. The data was painful but real. Built using only Pandas + NumPy - no fintech APIs, no ML."

Within 24 hours your post has 300+ likes, 50 comments, recruiters from Cred and Jupiter in your DMs, and three classmates asking if they can run it on their own data.

That is the project. That is the goal. Everything you do over the next seven days is in service of that one moment.

Why this works (the strategy)

Four reasons SpendDNA is designed to perform on GitHub in a way that no other project physically cannot:

- **It is personal - and universal.** Every Indian student uses UPI daily. When recruiters and peers see your analysis of your own UPI data, they immediately think "I want my own version." That curiosity is the engagement engine of the post.
- **It is visual.** The output is a clean, structured report with bars, percentages, archetype labels, and anomaly callouts. Same Spotify-Wrapped aesthetic that millions of people screenshot every December.
- **It mirrors real fintech work.** Transaction analytics, vendor categorisation, archetype detection, and anomaly flagging are core functions at every Indian fintech - Cred, Slice, Jupiter, Niyo, OneCard, Fi Money.
- **It defends against ChatGPT.** The vendor extraction patterns are specific to your dataset. The archetype assignment requires multi-step logic chains. The final insights cite specific numbers from this specific data. ChatGPT can help you write a groupby. It cannot tell you what percentage of Rahul's debit went to Swiggy.

STORY TIME

Why this hits LinkedIn harder than GroupDNA.

GroupDNA decodes social patterns, which is fun but mostly only relevant to the student's own friends.

SpendDNA decodes financial patterns, which is universally relevant. Every Indian on LinkedIn has a UPI app. Every recruiter, every senior engineer, every founder has thought about their spending. When they see your post, they will engage because they want their own version. Finance content also rides the strongest engagement algorithm on Indian LinkedIn, money is interesting to everyone. The combination of finance + Python + personal data is among the highest engagement content formats in the country.

2. The Problem Statement

You are given a raw bank/UPI transaction export - a CSV file containing 1,328 rows of date-stamped financial transactions for a fictional but realistic Bengaluru-based software engineer named Rahul Sharma, over six months (Jan to June 2024). Your job: parse this file, clean the messy fields, extract canonical vendor names from messy descriptions, categorise every transaction into one of 12 spending categories, detect Rahul's spending personality archetype, find anomalous transactions, and produce a comprehensive printed report.

The report must include – at minimum, eight clearly-labelled analytical sections, an Indian-fintech-style category breakdown, a monthly spending trend, time-of-day spending patterns, archetype labels, and an anomaly call-out section. The output should be printed to the console in a clean, formatted style that someone would want to screenshot.

Who is the user?

Imagine your end user is a 23-year-old engineering student or young professional in Bengaluru. They downloaded their HDFC statement for the last six months and now want to know, really know - where their money went. They want categories. They want a personality label. They want to be told whether they should be worried about their spending. They have zero patience for ugly output and zero tolerance for vague "you spent more on food this month" generalities.

Your code has one customer and her name is curiosity. Build for her.

3. Understanding the Input File

Real Indian bank statements are messy. SpendDNA's input is a CSV that replicates the messiness of an actual bank export. Here is a sample of what the lines look like:

```
Date,Time,Description,Type,Amount,Balance,Mode,Ref
12/04/24,21:14,UPI-SWIGGY-9876@HDFCBANK,DR,Rs.
487,52340.0,UPI,TXN123456 2024-04-12,21:18,BHIM
ZEPTO,Debit,₹628,51712.0,UPI,TXN837421
12-Apr-24,09:30,UPI-ZERODHA-COIN@AXIS,Debit,15000.00,36712.0,UPI,TXN9012
34 12 Apr 2024,14:48,UPI-ANKIT-2345@OKAXIS,DR,Rs.
500,36212.0,UPI,TXN556677
```

Edge cases you MUST handle

a. Four date formats

The Date column has FOUR different formats mixed together:

- "12/04/24" — DD/MM/YY (Indian short)
- "2024-04-12" — ISO format
- "12-Apr-24" — DD-Mon-YY
- "12 Apr 2024" — DD Mon YYYY

Use `pd.to_datetime(..., errors='coerce', dayfirst=True)`. It handles all four. Any row that can't parse → `NaT`, which you'll filter out at the end.

b. Three amount formats

The Amount column has multiple formats:

- "₹450" — with rupee symbol
- "Rs. 1,200" — with Rs. prefix and comma
- "1500.00" — plain decimal

Strip ₹, Rs., commas → `pd.to_numeric(errors='coerce')`. Same pattern you used in the Week 2 wrangling session.

c. Type field variants

The Type field is "DR"/"CR" or "Debit"/"Credit" — both formats appear in the same file. Standardise to a single canonical pair (we recommend lowercase "debit" and "credit" for downstream filtering).

d. Multi-variant vendor descriptions

This is the project's hardest cleaning challenge. The Description column has the actual merchant info buried inside UPI/POS/BHIM prefixes and suffixes. Examples from your dataset:

```
UPI-SWIGGY-1234@HDFCBANK → Swiggy
BHIM SWIGGY → Swiggy
POS SWIGGY BANGALORE → Swiggy
Swiggy*Order → Swiggy
BUNDL Tech P L → Swiggy (Swiggy's parent company!) POS ZOMATO
BANGALORE → Zomato
UPI-ZOMATO-LIMITED@PAYTM → Zomato
UPI-AMAZONPAY@HDFCBANK → Amazon
AMZN-INTPYMT → Amazon
```

You will build a vendor dictionary that maps keyword patterns to canonical vendor names. Inspect the descriptions first (`df['Description'].unique()`) – there are dozens of variants). Then build the mapping.

e. P2P transfers and ATM withdrawals

Some transactions are not merchant payments – they are friend-to-friend transfers (e.g. UPI-PRIYA 9876@OKAXIS) or ATM withdrawals (ATM-WDL-HDFC-1234). These get a special category of "Personal Transfer" or "Cash Withdrawal" rather than being categorised by vendor.

f. Duplicates and noise

There are 18 exact duplicate rows in the file. Drop them. Some Mode values are empty strings – treat as missing.

4. The Provided Dataset - rahul_transactions.csv

You will work with a single CSV that we have prepared specifically for this project. It is synthetic — generated by us - but built around realistic Indian transaction patterns. The persona behind the data is a fictional Bengaluru software engineer.

Persona profile

The transactions belong to:

Name	Rahul Sharma
Age	24
Role	Software Engineer at a Bengaluru product startup
Monthly salary	~₹85,000 credited on the 1st
Monthly rent	₹18,000 debited on the 3rd-5th
Monthly SIP	₹15,000 to Zerodha on the 7th
Opening balance	₹4,00,000 (his accumulated savings)

Dataset stats

File name	rahul_transactions.csv
Format	CSV, UTF-8
Period	01 January 2024 to 30 June 2024 (6 months)
Total rows	1,328 (including 18 duplicates)
Columns	8 (Date, Time, Description, Type, Amount, Balance, Mode, Ref)
Unique vendors	~35 canonical vendors across 12 categories
Anomaly transactions injected	~30 (3-5x typical category amount)

Why a synthetic dataset

- **Privacy.** You don't share your real bank statement with anyone. Including us.
- **Consistency.** Every student gets the same dataset. Expected outputs are gradable.
- **Anti-plagiarism.** Any code copied from a generic GitHub transaction analyzer will fail on our specific vendor variants and date formats. Submissions must run on rahul_transactions.csv.
- **Engineered for detection.** The patterns are deliberately strong - Food Delivery is ~23% of debits, Quick Commerce is ~20%, Investments are ~16%. Anomalies are ~3-5x typical amounts and easy for a z-score to catch. If your archetype detection doesn't flag Rahul as a Foodie + Investor, your logic has a bug.

Running on your own data (bonus)

After your code works on the provided dataset, you are encouraged - not required, to export your own bank or UPI statement and run the analysis on it. This is the step that turns into your LinkedIn post. Section 11 covers the exact strategy.

Important: do NOT submit your own personal data as the assignment. Submit only the notebook running on rahul_transactions.csv. Your own data is for the bonus screenshot only.

INDUSTRY CONNECT

What this dataset teaches you about real-world data.

Real fintech data does not arrive clean. It has 4 date formats mixed in one column. Vendor names buried inside UPI prefixes. Amounts as strings with multiple currency notations. Duplicates. Missing modes. Friend-to-friend transfers that look like vendor payments.

Every transaction analytics team at Cred, Slice, Jupiter, Niyo, and PhonePe spends at least half their time cleaning data exactly like this. The ability to look at a raw transaction file and build a robust parser is the single most-valued skill in fintech analytics.

When you describe this project in an interview at a fintech, the sentence that lands is: "I wrote a vendor-extraction system that handles 5+ format variants per merchant and 4 date formats in one column." That sentence beats "I built a calculator." Every single time.

5. New Concepts You Need

Week 2 covered everything you need except two pandas tools. Both are tiny additions, both are core fintech analytics building blocks.

5.1 The .dt accessor

Once you've converted your Date column to datetime, you can extract individual time components with the .dt accessor:

```
df['date'] = pd.to_datetime(df['Date'], errors='coerce', dayfirst=True)
df['hour'] = df['Time'].str[:2].astype(int) # from the Time column
df['month'] = df['date'].dt.month
df['day_of_week'] = df['date'].dt.day_name() # 'Monday', 'Tuesday', ...
df['week_number'] = df['date'].dt.isocalendar().week
```

These are the only datetime methods you need. Together they unlock monthly trends, day-of-week analysis, and time-of-day patterns, all core SpendDNA features.

5.2 Z-score anomaly detection

A z-score tells you how many standard deviations a value is from the mean. A z-score above 2 means "this value is in the top 2.3% of the distribution" - i.e., unusually high. We use this to flag anomalous transactions in Feature 7.

```
category_mean =
df.groupby('category')['amount'].transform('mean') category_std
= df.groupby('category')['amount'].transform('std')
df['z_score'] = (df['amount'] - category_mean) / category_std
anomalies = df[df['z_score'] > 2] # transactions 2+ stddev above category mean
```

This is a fundamental statistical tool used in fraud detection, outlier removal, and anomaly flagging. You'll see it in every real fintech analytics codebase.

6. Core Features – What You Must Build (Mandatory)

Eight features below. All mandatory. Each one is described with: what it does, why it matters, which concepts you exercise, hints where needed, and expected output.

Feature 1 - The Transaction Parser

What it does. Reads `rahul_transactions.csv`. Parses the four date formats. Parses the three amount formats. Standardises the Type field. Drops duplicates. Stores everything in a clean DataFrame ready for analysis.

Why it matters. Foundation. Every feature depends on this. A wrong parser kills the whole project.

Concepts used. `pd.read_csv`, `pd.to_datetime` with `errors='coerce'`, `.str.replace`, `pd.to_numeric`, `drop_duplicates`, `.str.lower`.

Hint. For dates, use `dayfirst=True` so `'12/04/24'` is read as 4th December, not 12th April. For amounts, `chain(.str.replace('₹', ''), .str.replace('Rs.', ''), .str.replace(',', ''))` then `pd.to_numeric`.

Expected outcome. Print: 'Parsed 1310 transactions across 6 months. Dropped 18 duplicates. 0 unparseable amounts, 0 unparseable dates.'

Feature 2 - Vendor Extractor

What it does. From each messy Description string, extract the canonical vendor name. Build a vendor dictionary mapping keyword patterns to canonical names. Add a `vendor_clean` column to the DataFrame.

Why it matters. This is the project's signature feature. Real fintech analysts call this 'merchant normalisation' and it is core to every transaction-analysis pipeline. Without it, you cannot categorise — and without categorisation, the entire downstream analysis collapses.

Concepts used. `.str.contains` with `case=False`, dictionaries, loops, conditional logic. No regex.

Approach. Step 1: `print df['Description'].unique()` and inspect what's there. Step 2: build a dictionary like `{ 'Swiggy': ['SWIGGY', 'BUNDL'], 'Zomato': ['ZOMATO'], 'Amazon': ['AMAZON', 'AMZN'], ... }`. Step 3: write a function that takes a description, checks each canonical name's keywords against it (case-insensitive), returns the first match. Step 4: apply to the Description column.

Tricky cases. (a) Swiggy descriptions sometimes say 'BUNDL Tech P L' - that's Swiggy's parent company name on official bank statements. You need to know to include 'BUNDL' as a Swiggy keyword. (b) Friend transfers like 'UPI-PRIYA-9876@OKAXIS' should map to a special vendor 'P2P Transfer', not be left uncategorised. (c) ATM withdrawals like 'ATM-WDL-HDFC-1234' should map to 'Cash Withdrawal'.

Expected outcome. After running, `print df['vendor_clean'].nunique()` - should be ~35 canonical vendors.

Print `df['vendor_clean'].value_counts().head(10)` to see top vendors.

Feature 3 - Category Tagger

What it does. Maps every vendor to one of 12 categories: Food Delivery, Quick Commerce, E commerce, Transport, Cafe, Restaurants, Subscriptions, Utilities, Groceries, Investments, Fuel, Entertainment, plus Personal Transfer and Cash Withdrawal.

Why it matters. Categories are the unit of insight. "Rahul spent ₹15,000 on Swiggy" is interesting. "Rahul spent 23% on Food Delivery" is actionable. Categories tell stories.

Concepts used. Dictionary mapping, `.map()` with NaN handling for uncategorised entries.

Expected outcome. Every transaction has a category. Verify by printing `df['category'].value_counts()` - Food Delivery should be the top category by frequency, with ~280-350 transactions over 6 months.

Feature 4 - Spending Overview

What it does. Computes and prints the headline financial stats: total credits, total debits, net savings, savings rate, top 5 categories by spend, top 5 vendors by spend, total transaction count.

Why it matters. This is the executive summary at the top of your final report. Anyone scanning the output should know within 5 seconds whether this person is a saver or a spender.

Concepts used. `groupby + sum`, `sort_values`, percentages with f-string formatting.

Expected output. Numbers should roughly match: total credits ~₹5.1L, total debits ~₹8.2L, savings rate ~ -62% (he's an over-spender). Top category: Food Delivery.

Feature 5 - Monthly Trend Analysis

What it does. Build a NumPy/Pandas matrix: rows = categories, columns = months (Jan, Feb, Mar, Apr, May, Jun), values = total spent per category per month. Identify which category had the BIGGEST month on-month growth (a 'trending up' signal) and which had the biggest decline.

Why it matters. Monthly trends are how Cred and Jupiter present spending to their users. You want to know not just where your money went, but where it's going faster or slower.

Concepts used. `.dt.month`, `pivot_table` or `groupby` with `unstack`, NumPy for percentage change computation.

Hint. `month_pivot = df.pivot_table(values='amount', index='category', columns='month', aggfunc='sum')`. To find biggest growth, compute $(\text{last_month} - \text{first_month}) / \text{first_month} * 100$ for each category.

Feature 6 - Time-of-Day Patterns

What it does. Analyse when Rahul spends. Build a heatmap-style matrix of category x hour-of-day. Surface insights like: 65% of Swiggy orders happen between 9 PM and 1 AM. Cafe spending is

concentrated 8-11 AM (morning coffee runs).

Why it matters. Time-of-day patterns reveal real behaviour. Late-night food orders signal stress or single-living. Morning cafe spend signals commute habits. These are the patterns fintech apps surface to drive 'lifestyle insights'.

Concepts used. .dt.hour or string slicing on the Time column, NumPy 2D array (categories x 24), printable bar visualization.

Hint. Use the Time column (already in HH:MM format). Extract hour with `df['Time'].str[:2].astype(int)`. Build the matrix using `groupby` on (category, hour) or with NumPy.

Feature 7 - Anomaly Detection

What it does. Find transactions that are anomalously large for their category. Use z-score: for each category, compute mean and std; flag transactions where `z_score > 2` (top ~2% of category).

Why it matters. Anomalies tell the most interesting stories — the one time Rahul spent ₹12,000 at Myntra, or the ₹8,000 BookMyShow order. These are the 'huh, what happened there?' moments. Real fintech apps surface these in monthly recaps.

Concepts used. `groupby + transform('mean')`, `groupby + transform('std')`, boolean filtering, sort by `z_score` descending.

Hint. Use the exact z-score formula in Section 5.2 of this brief. Print the top 5 anomalies with date, vendor, category, amount, `z_score`.

Expected outcome. Should surface ~25-30 anomalies. Top anomalies are usually in E-commerce and Restaurants categories where a few transactions are 3-5x the typical amount.

Feature 8 - Spending Archetype Detection

What it does. Apply rules from Section 7 to assign Rahul to spending archetypes. One transaction history can match MULTIPLE archetypes - print them all, with the supporting metric for each.

Why it matters. Archetypes are the LinkedIn-screenshot moment. "This user is The Foodie + The Investor + The Late-Night Snacker." These labels are what students will share.

Concepts used. Functions (one per archetype detection rule, ideally), conditionals, the metrics already computed in Features 4-7.

7. The Spending Archetype System

Eight archetypes, each with a quantitative detection rule. Apply ALL rules to Rahul's data - he can match multiple archetypes. The 'Should Apply To Rahul?' column tells you whether your code should flag this archetype on the provided dataset (this is your bug-checker).

Archetype	Detection Rule	Should Apply To Rahul?
THE FOODIE	Food Delivery + Restaurants + Cafe combined > 25% of	YES (about 30%)

	debits.	
THE QUICK COMMERCE JUNKIE	Quick Commerce > 15% of debits.	YES (about 20%)
THE SHOPAHOLIC	E-commerce > 15% of debits.	YES (about 17%)
THE INVESTOR	Investments > 15% of debits.	YES (about 16%)
THE LATE-NIGHT SNACKER	More than 50% of Food Delivery transactions happen between 21:00 and 02:00.	YES
THE CAB COMMUTER	Transport > 10% of debits.	NO (Rahul transport is about 2%)
THE SUBSCRIPTION LOVER	5 or more distinct subscription vendors active.	YES (4-5 active)
THE YOLO SPENDER	Savings rate (credits debits)/credits is below 10%.	YES (savings rate is negative)
THE DISCIPLINED SAVER	Savings rate above 40%.	NO

Rahul should match around five of these. The bonus angle: invent your own ninth archetype - one specific to Indian college life or Bengaluru tech-bro culture (suggestions: "The Brewery Regular", "The Pavement Coffee Connoisseur", "The Crypto Curious"). Define your detection rule clearly. +2 marks for one well-defended invented archetype.

8. Bonus Features

- **Run on your own statement (+5 marks).** Export 3-6 months of your own UPI/bank statement (most banks let you download a CSV). Apply your code. Take a screenshot of the output. Add it to your notebook as the final cell. Do NOT push your raw statement to GitHub - only the screenshot.
- **Invented archetype (+2 marks).** Define one new archetype with a quantitative detection rule that fits Indian/Bengaluru context.
- **Day-of-week analysis (+3 marks).** Extend Feature 6 to also analyse spending by day-of-week. Are weekends 50% more expensive than weekdays? Surface this.
- **Vendor cleanup audit (+3 marks).** Print a list of any descriptions in the file that your vendor extractor failed to map (returned 'Uncategorised'). A well-built extractor should have under 5 such cases. Show them in your notebook.
- **Spend forecasting (+5 marks, ambitious).** Using only NumPy: compute the rolling 3-month average spend per category and project the next month's spend as that average. No ML. No statsmodels. Just NumPy and arithmetic.

9. Tech Stack - What You Can and Cannot Use

The strict library list matters. Constraint discipline is what makes this project respectable to a recruiter. Anyone can throw pandas-profiling at a CSV. Few can build polished analytics with restraint.

YOU CAN USE	YOU CANNOT USE
• All Python fundamentals (variables, types, conditionals, loops, functions, f-strings, split, map, lambda) • Lists, tuples, sets, dictionaries, comprehensions • NumPy (np.array, np.zeros, np.ones, np.arange, mean, std, sum, max, min, percentile) • Pandas (read_csv, DataFrame, Series, groupby, agg, pivot_table, transform, apply, .dt, .str) • datetime module (datetime.strptime if needed alongside pd.to_datetime) • All string methods you have learned • pd.cut, pd.qcut • sorted() with key= and reverse=	• pandas-profiling, ydata-profiling, sweetviz, autoviz • matplotlib, seaborn, plotly (your visualisation is text/ASCII-based) • scikit-learn, scipy.stats, statsmodels — z-score formula by hand only • collections.Counter — build your own counter with a dict • re (regex) — all pattern matching via string methods • Any transaction parsing library (banking-csv parser, plaid, etc) • Any AI / ML / GenAI library • Any external transaction datasets — use rahul_transactions.csv only

The forbidden list is the project's point. Anyone can produce a transaction analysis with pandas-profiling in 5 lines of code. You are demonstrating that you can build the analysis yourself - which is the skill recruiters actually pay for.

10. The 7-Day Build Plan

Do not attempt this in one sitting. Follow this plan day by day.

Day 1 - Parser + Setup (2 hours)

- **Goal:** Read the CSV. Convert dates (4 formats) and amounts (3 formats). Standardise Type. Drop duplicates. Store cleaned data as df_clean.
- **Checkpoint:** df.dtypes shows date as datetime64, amount as float64. df.shape is around (1310, 8) after dedup.

Day 2 - Vendor Extractor (3 hours)

- **Goal:** Inspect df['Description'].unique(). Build the vendor dictionary. Write extract_vendor function. Apply to the DataFrame. Verify with value_counts().
- **Checkpoint:** df['vendor_clean'].nunique() shows around 35. Top vendors: Swiggy, Zomato,

Zepto, Amazon, etc.

Day 3 - Category Tagger + Spending Overview (1.5 hours)

- **Goal:** Map vendors to 12 categories. Compute totals, top vendors, top categories. Print the executive summary.
- **Checkpoint:** Total credits around 5.1L, total debits around 8.2L. Food Delivery should be the top category.

Day 4 - Monthly Trends + Time-of-Day (2 hours)

- **Goal:** Use .dt.month to compute monthly totals per category. Use Time column to compute hour-of-day. Identify late-night spending.
- **Checkpoint:** More than 50% of Food Delivery transactions are between 9 PM and 1 AM. If your code doesn't surface this, your hour extraction has a bug.

Day 5 - Anomaly Detection + Archetype Detection (2 hours)

- **Goal:** Compute z-scores per category. Flag $z > 2$. Implement archetype rules. Apply to Rahul.
- **Checkpoint:** Around 25-30 anomalies flagged. Archetypes detected: Foodie, Quick Commerce Junkie, Shopaholic, Investor, Late-Night Snacker, YOLO Spender.

Day 6 - Final Printed Report (2 hours)

- **Goal:** Pull all findings into one beautifully formatted printed report. Use ASCII bars, aligned columns, section dividers.
- **Checkpoint:** Your report looks like Section 11 below. If you wouldn't screenshot it, format isn't done.

Day 7 - Polish + Insights + Submit (1 hour)

- **Goal:** Three data-specific insights in markdown. Reflection cell. AI-assistance disclosure. Push to GitHub or Colab. Submit link.

11. The Target Printed Report

Below is the shape your final output should take. The numbers shown are illustrative — yours will be close (within 5%) if your analysis is correct.

=====

== SpendDNA REPORT - RAHUL SHARMA

6 months - 1,310 transactions - Jan to Jun 2024

=====

EXECUTIVE SUMMARY

Total credits : Rs. 5,09,774
Total debits : Rs. 8,26,679
Net change : -Rs. 3,16,905 (overspending)
Savings rate : -62.2% (BURNING SAVINGS)
Transactions : 1,310
Unique vendors : 35

TOP CATEGORIES (% of debit total)

Food Delivery ##### 23.0% Rs. 1,89,899
Quick Commerce ##### 19.5% Rs. 1,61,564
E-commerce ##### 16.8% Rs. 1,39,018
Investments ##### 16.4% Rs. 1,35,821
Restaurants ##### 5.1% Rs. 42,125

TOP VENDORS

Swiggy Rs. 1,12,432 (294 orders)
Zomato Rs. 77,467 (180 orders)
Amazon Rs. 78,541 (48 orders)
Zerodha Rs. 90,000 (6 SIPs)
Zepto Rs. 62,318 (108 orders)

TIME-OF-DAY PATTERNS

Food Delivery peaks: 21:00 - 01:00 (62% of orders)
Cafe peaks: 09:00 - 11:00 (morning runs)
Quick Commerce: evenly distributed

MONTHLY TREND (Food Delivery)

Jan Rs. 28,432 #####
Feb Rs. 31,108 #####
Mar Rs. 36,212 #####
Apr Rs. 34,221 #####
May Rs. 32,156 #####
Jun Rs. 27,770 #####

TOP ANOMALIES (3+ stddev from category mean)

14 Apr - Myntra Rs. 12,540 (z=4.2)
22 Mar - Amazon Rs. 18,300 (z=3.8)

```
08 May - BookMyShow Rs. 5,800 (z=3.2)
```

RAHUL'S SPENDING ARCHETYPES

- > THE FOODIE (30.1% on food)
- > THE QUICK COMMERCE (19.5% on Q-com)
- > THE SHOPAHOLIC (16.8% on e-commerce)
- > THE INVESTOR (16.4% on SIPs)
- > THE LATE-NIGHT SNACKER (62% food after 9 PM)
- > THE YOLO SPENDER (savings rate -62%)

== KEY INSIGHTS

1. Rahul is burning through his savings at Rs. 53,000 per month - 62% over his salary. Unsustainable past Q3 2024.
2. 70% of his food-delivery spend happens after 9 PM suggesting stress eating or single-living patterns.
3. Investments (16.4%) are healthy, but discretionary (Food + Q-com + E-com = 59%) dominates his wallet.

12. The Showcase - LinkedIn & GitHub Strategy

Building the project is half the job. Showcasing it is the other half. Skip the showcase and the recruiter never sees it.

Step 1 - Run it on your own statement (the secret weapon)

After your code works on rahul_transactions.csv, export your own 3-6 month bank/UPI statement (HDFC, ICICI, SBI YONO, Paytm - all support CSV exports). Save it as my_statement.csv. Run your code. Take a screenshot of the printed report. This is the post.

CRITICAL - do not push your real statement to GitHub. Only the screenshot. Your raw data stays on your laptop.

Step 2 - Write the LinkedIn post

Template:

I spent a weekend building a Python tool that analyzes 6 months of my UPI transactions.

Some findings about my own spending were... uncomfortable:

- [X]% of my spend went to [top category]
- [Y]% of my food-delivery orders happen after 9 PM
- I spend roughly Rs. [Z] / month on Quick Commerce
- My personality archetype: THE [ARCHETYPE]

Built using only Python fundamentals, NumPy, and Pandas. No fintech APIs. No ML. No matplotlib. Just clean analytics on messy bank-export data - the kind of work transaction-analytics teams at Cred / Slice / Jupiter do every day.

Code on GitHub: [link]

#DataAnalytics #Python #Pandas #Fintech #UnloxAcademy

Step 3 - GitHub README

Your repo should have one notebook (.ipynb), the synthetic dataset (rahul_transactions.csv), and a README.md including: project title, one-line description, output screenshot (essential), constraint list (what you allowed vs forbade), and how to run.

Step 4 - Pin it

On LinkedIn, pin the post to your profile. On GitHub, pin the repo. Both take two clicks. The difference between unpinned and pinned is the difference between a recruiter scrolling past and stopping at it.

INDUSTRY CONNECT

Why this post outperforms calculator projects 50:1.

We have measured engagement on student-built projects posted by past Unlox cohorts. Generic projects (calculators, expense trackers) average 20-30 likes. Personal-data projects (GroupDNA, SpendDNA-style) average 250-400 likes. The reason is simple: every reader of the personal-data post immediately wants their own version. Every reader of the calculator post thinks "I built that in college."

If you do nothing else this week, finish the project, run it on your own data, post the screenshot. Three actions. Six hours total. Worth more than the entire semester of unfinished projects on your CV.

13. Evaluation Rubric

Total: 100 marks. Pass: 65. Distinction (Featured Project on cohort wall): 85+. Below 65 = re-submission required.

Parser (F1)	10 marks. All 4 date formats, all 3 amount formats, Type standardisation, duplicates dropped.
Vendor Extractor (F2)	20 marks. Around 35 canonical vendors. Less than 5 'Uncategorised'. Tricky cases (BUNDL, P2P, ATM) handled.
Category Tagger (F3)	10 marks. All 12 categories used. Verified with value_counts.
Spending Overview (F4)	10 marks. Numbers match (within 5%) expected values in Section 4.
Monthly Trends (F5)	10 marks. Correct pivot or groupby. Trend direction visible.
Time-of-Day (F6)	10 marks. Hour extracted correctly. Late-night Food Delivery surfaced.
Anomaly Detection (F7)	10 marks. z-score formula correct. Top anomalies match expected pattern.
Archetype Detection (F8)	10 marks. All 5-6 expected archetypes detected for Rahul. Logic separated into functions.
Final Output + Insights	5 marks. Formatted report. Three data-specific insights.
Code quality + README	5 marks. Readable variable names, comments, README with screenshot.

Constraint violation = automatic zero on the affected feature. Using regex = zero on Vendor Extractor. Using matplotlib = zero on Time-of-Day. The whole point is constraint discipline.

14. Submission

- **File:** One .ipynb. File name: SpendDNA_<YourName>.ipynb
- **Upload to:** Google Colab (Anyone with link can view) OR public GitHub repo. •

Submit: Only the LINK via the Google Form.

- **Must include:** Markdown cell at top with project title, your name, batch, date. Each Feature in its own labelled section. Final reflection cell.
- **Notebook MUST run end-to-end:** from cell 1 to last cell, without errors, when rahul_transactions.csv is in the same folder or uploaded to Colab session.
- **Deadline:** 7 days from today.

Anti-plagiarism

All submissions go through a code-similarity checker. Two submissions more than 70% similar in structure and variable naming = both get zero. AI-generated cells are permitted as a learning aid; the final submission must be in your own structure. If a cell is AI-assisted, mark it with: # AI-assisted: <one line description>. Honest disclosure is rewarded. Silent copy-paste is caught.

15. Frequently Asked Questions

Q: Can I use pandas-profiling for the analysis?

No. The whole point is to do the analysis yourself. pandas-profiling is for prototyping, not for building shippable analytics. A recruiter who sees pandas-profiling in your notebook will assume you don't know how to do real analysis.

Q: My vendor extractor labels some transactions as 'Uncategorised'. Is that wrong? Under 5 such cases is fine — there will always be edge cases in real data. Over 5 means your dictionary is incomplete. Inspect `df[df['vendor_clean']=='Uncategorised']['Description'].unique()` and add more keyword mappings.

Q: Should P2P transfers count as 'spend'?

Depends on interpretation. We recommend keeping them in their own category 'Personal Transfer' and excluding them from the category percentage calculations (they're not 'consumption'). But document your choice in a comment.

Q: My anomaly detector flags 50+ transactions. That seems too many.

Likely you computed z-score across ALL transactions instead of WITHIN each category. The correct approach uses groupby + transform to compute mean/std per category, so a ₹2000 Swiggy order is anomalous within Food Delivery but a ₹2000 Amazon order is normal within E-commerce.

Q: Can I add my own archetype beyond the 8 listed?

Yes. One invented archetype with a clear quantitative detection rule, defended in comments, earns +2 marks. Suggestions: 'The Brewery Regular' (Friday-evening cafe/restaurant spend), 'The Subscription Hoarder' (8+ active subs), 'The Pavement Coffee Connoisseur' (Cafe spend at 4+ unique vendors). Make it Bengaluru-specific.

Q: How do I extract hour from the Time column?

`df['hour'] = df['Time'].str[:2].astype(int)`. The Time column is already in HH:MM format so the first 2 characters are always the hour. No need to use datetime for this.

Q: My total credits and debits don't exactly match Section 4 numbers.

Slight differences (within 5%) are fine — they depend on your exact handling of duplicates, P2P transfers, and ATM withdrawals. Off by more than 10% = bug in parser or filter. Most common bug: dropped salary credits because the type field standardisation didn't catch a variant.

Q: Can I use ChatGPT for this project?

Yes, with disclosure. ChatGPT cannot tell you which percentage of Rahul's spending went to which category — only you can, by running the code. Use it for: syntax help, debugging, code review. Do not

use it for: building the vendor dictionary (it doesn't know your variants), writing the final insights (must be data-specific). Disclose any AI-assisted cell with a comment.

Q: My output formatting looks ugly.

Spend Day 6 on formatting. It is graded. Use f-strings with width specifiers ('{name:<15}'), use '=' * 60 for dividers, align columns. A clean report is worth 5 marks. An ugly report loses you those 5 marks AND kills your LinkedIn post.

Q: Can I do bonus features and skip a core one?

No. Bonus features add marks ON TOP of the core 100. Skipping a core feature loses you its marks AND any bonus marks become irrelevant because we cap at 100.

16. A Note Before You Start

Of all the projects we have designed in the DA/DS curriculum, this is the one we are most excited about. SpendDNA combines genuine industry relevance (transaction analytics is a real fintech job category) with strong personal hooks (every Indian student uses UPI) and strong LinkedIn-shareability (Spotify Wrapped-style outputs are a proven engagement format).

When you finish this project and run it on your own bank data, you will see your own financial behaviour in a way no app has shown you before. You may be uncomfortable with what you find. That discomfort is the project's secret value — it is also why you will post it on LinkedIn. Authentic discomfort is more engaging than fake success.

When you are stuck on Day 3 or Day 4 (and you will be — vendor extraction is genuinely hard), remember: the students who panic when they hit a wall are the ones who never build anything memorable. The students who sit with the problem for an hour, walk away, come back, and try again — those are the students who post the screenshot at the end and get the recruiter DMs.

Treat this project like your first real fintech product. The user is real (you). The output is real (a screenshot you will share). The constraint is real (no shortcuts). Build it that way.

Now go decode your wallet.